

What Makes Games Popular? An Analysis on the Features of Video Games

Alexander Zhong

Department of Computer Science
Stanford University
ahzhong@stanford.edu

Kyle Huang

Department of Electrical Engineering
Stanford University
kyleh1@stanford.edu

1 Introduction

The gaming industry is one of the largest global entertainment markets, with thousands of new titles each year and billions in annual revenue. For developers, publishers, and marketers, understanding what drives a game's success is crucial for budgeting, feature design, and marketing, yet most work relies on simple statistics rather than machine learning. In this project, we study how well game metadata can predict and explain the popularity of Steam games. For each game, we construct a feature vector with fields such as whether the game is free, release year, price and initial price, discount, number of achievements, description length, genres, categories, supported platforms and languages, developer, and publisher. Using linear regression and a neural network, we predict a composite popularity score that combines log-transformed owner counts, lifetime and recent playtime, review sentiment, normalized Metacritic score, and discount percent. With the same inputs, we also apply k-means and Gaussian Mixture Models to cluster games into latent groups (for example, indie hits versus AAA titles) and analyze which feature patterns characterize clusters. Our goal is both to build accurate predictors of this popularity metric and to uncover interpretable structure that helps explain what makes games successful.

2 Related Work

Most prior work on game popularity treats it as a supervised prediction problem on tabular features. For example, Lukáš Kříž and Vašíček (2024) apply machine learning methods to forecast the number of video game players from Steam-related features, demonstrating that relatively simple models can achieve good predictive performance but offering limited insight into latent structure among games. Jiang (2021) focus specifically on independent titles on Steam and study how metadata such as genres, tags, and reviews relate to subsequent popularity, while Luisa et al. (2021) use Bayesian regression and a genre-based hierarchical model to predict future player counts and interpret the influence of price, languages, and other properties. Beyond these Steam-focused studies, Li et al. (2021) and Hu et al. (2024) analyze larger multi-platform datasets and report that tree-based ensembles and gradient boosting methods (e.g., Random Forest, XGBoost, LightGBM), sometimes combined with hybrid feature selection, often outperform simpler linear models for sales or popularity prediction. Survey work such as Chertov and Buslaiev (2025) emphasizes the growing role of social-media-derived features, while Horsfall and Oikonomou (2011) investigate how specific game-play and design aspects affect the popularity of role-playing games using more traditional statistical analyses on smaller curated datasets. In contrast to these approaches, our work combines a supervised baseline (linear regression) and a neural network with unsupervised methods (k-means and GMM) on the same Steam feature space, with the goal of not only predicting popularity but also uncovering interpretable clusters and recurring feature patterns; in practice, many developers still rely on hand-crafted heuristics and domain intuition, so we view our approach as a step toward more systematic, data-driven understanding of what makes games popular.

3 Dataset and Features

Our dataset consists of approximately 27,000 Steam games and their associated metadata. We started from a base Kaggle dataset of Steam store games Davis (2021) and enriched it by querying the Steam Web API Corporation and related developer endpoints SteamSpy to obtain additional fields such as

metacritic scores, owner counts, and playtime, joining them by steam ID. After filtering games with missing or invalid identifiers and names, the combined dataset was converted into a structured design matrix. Each game is represented as a feature vector of 513 dimensions. List-based fields such as genres, categories, platforms, languages, developers, and publishers were standardized by building vocabularies over the data and encoding each example as multi-hot indicators. Numeric attributes such as price and initial price (in USD), discount, total achievements, and description length were cleaned using safe numeric parsing, date parsing, HTML stripping, and general text cleaning. We then generated a composite popularity score as the regression target by combining several engagement and reception signals: log-scaled owner counts, lifetime and recent playtime (including recent peak concurrent users), the ratio of positive to total ratings, normalized Metacritic score, and discount fraction, with fixed weights on each component. For the feature matrix, we applied column-wise standardization to all numeric features while preserving the structure of the one-hot encodings, which produced a clean and consistent input representation for all models. For supervised experiments, we split the data into a 70% training set and a 30% validation set using a random train/validation split, and we did not use additional data augmentation. For some clustering experiments we also applied PCA to reduce the standardized features to 50 dimensions for efficiency, while all regression models were trained on the full 513-dimensional feature space.

4 Methods

Linear regression: We use ℓ_2 -regularized linear regression as a baseline. This gives a simple, interpretable baseline for our popularity score.

Neural network: We implement a fully connected feedforward network in PyTorch to capture nonlinear relationships in our data.

K-means clustering: To discover coarse groups of games, we apply k-means to the standardized feature matrix.

Gaussian Mixture Models: For probabilistic clustering with soft assignments, we fit Gaussian Mixture Models (GMMs) with K components.

Model selection and clustering metrics: The number of clusters k and clustering quality are evaluated using criteria not covered in class. For k-means, we inspect inertia curves (Elbow Method) as a function of k . For GMMs, we select k by minimizing the Bayesian Information Criterion in (1), which trades off fit and model complexity. Cluster compactness and separation are measured by the silhouette coefficient in (2), where $a(i)$ is the mean distance from point i to points in its own cluster and $b(i)$ is the smallest mean distance to any other cluster. For stability, we compute the Adjusted Rand Index (ARI) between pairs of clusterings, which measures agreement between partitions while correcting for chance (values near 1 indicate very similar clusterings, values near 0 correspond to random agreement).

$$\text{BIC} = \ln(m) d - 2 \ln(\hat{L}) \quad (1) \quad s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (2)$$

5 Experiments / Results / Discussion

5.1 Regression Models for Popularity Prediction

5.1.1 Linear Regression

Using the processed dataset, we trained a linear regression model with L_2 regularization and evaluated performance using MSE on the validation set. We experimented with several learning rates between 0.1 and 0.001 and found that 0.1 produced the most stable and fastest convergence. We also varied the regularization strength and observed the bias variance trade off and found it consistent with theory: small amounts of regularization reduced overfitting and improved performance, while larger values caused underfitting and increased error. The best performance across our runs was a training MSE of 0.478 using a learning rate of 0.1 and a regularization parameter of 0.1.

5.1.2 Neural Network

We implemented a neural network using a 70/30 train-test split (due to a large dataset) and varied several hyperparameters, including learning rate, number of hidden layers, and number of neurons per layer, to evaluate their impact on validation MSE. Learning rate had the largest effect on convergence:

high rates (e.g., 0.1) frequently caused divergence or poor performance, while very low rates (<0.001) dramatically slowed convergence without improving accuracy. We also observed that training past the point of minimum validation loss led to increasing validation error, indicating overfitting.

With tuned hyperparameters, the neural network consistently outperformed linear regression, achieving a best validation MSE of about 0.47 compared to 0.48 for linear regression, though the improvement was modest, suggesting that our data exhibits some but limited nonlinearity. Increasing network depth did not provide meaningful gains and substantially increased training time, and a shallow architecture with a single hidden layer is sufficient.

Finally, we compared ReLU and LeakyReLU activations and found no measurable performance difference. Overall, a learning rate of 0.01 produced the most stable and effective results.

Learning Rate	Activation	Hidden Layers	Neurons	Validation MSE	Convergence Step
0.1	ReLU	1	(64)	0.480	185
0.01	ReLU	1	(64)	0.471	145
0.001	ReLU	1	(64)	0.473	500+
0.1	ReLU	2	(64, 32)	0.717	100
0.01	ReLU	2	(64, 32)	0.470	165
0.001	ReLU	2	(64, 32)	0.472	500+
0.1	ReLU	3	(128, 64, 32)	0.712	175
0.01	ReLU	3	(128, 64, 32)	0.470	160
0.001	ReLU	3	(128, 64, 32)	0.472	260
0.1	ReLU	3	(256, 128, 64)	0.868	500+
0.01	ReLU	3	(256, 128, 64)	0.473	100
0.001	ReLU	3	(256, 128, 64)	0.474	100
0.01	LeakyReLU	1	(64)	0.472	120
0.01	LeakyReLU	2	(64, 32)	0.474	70
0.01	LeakyReLU	3	(128, 64, 32)	0.470	160
0.01	LeakyReLU	3	(256, 128, 64)	0.469	140

Table 1: Neural network hyperparameter tuning.

LR	Reg	Val Loss	Conv.
0.1	0	0.685	564
0.1	0.1	0.478	280
0.1	1	0.515	37
0.01	0	0.927	1000+
0.01	0.1	0.515	1000+
0.01	1	0.515	335
0.001	0	11.468	1000+
0.001	0.1	7.233	1000+
0.001	1	0.755	1000+

Table 2: Linear regression hyperparameter tuning.

5.2 K-Means and Gaussian Mixture Model to Find Suitable Clusters

To better understand whether our feature space contains meaningful underlying clusters and which features characterize them, we applied both k-means and GMMs to our data.

5.2.1 K-Means

First, we selected a value of k using both Elbow and Silhouette Methods. We ran k-means for $k \in [3, 30]$ and recorded inertia and the average silhouette score. Initially, the silhouette score increased from $k = 3$ to a peak (0.43) at $k = 11$ before dropping sharply. This peak matched the inertia curve, which flattened around $k = 10, 11$, so we initially chose $k = 11$. However, manual inspection revealed that several clusters were driven primarily by missing data (e.g. a cluster consisting mostly of games with missing genre and category). After improving our preprocessing by using more consistent defaults and fill values for missing entries and rerunning analysis, the silhouette scores showed a weaker peak around $k = 4 - 7$, while the elbow in the inertia curve appeared roughly between $k = 7 - 13$.

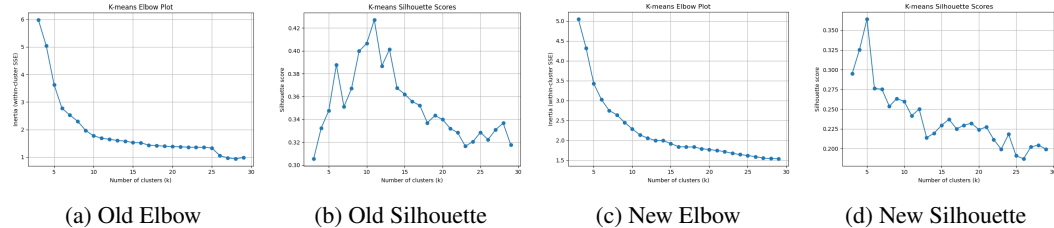


Figure 1: K-means comparison plots

To refine our choice of k , we focused on the range $k \in [4, 10]$ and evaluated both variability in inertia and the Adjusted Rand Index (ARI) between runs. For each value of k , we ran k-means 20 times with different random initializations on the standardized features. Among these candidates, $k = 7$ had the highest average ARI and relatively low variability in inertia, so $k = 7$ was selected for further analysis. This indicates that many initializations converge to similar partitions of the data, while a minority of runs find alternative local optima with slightly different cluster assignments. Overall, the

clustering structure at $k = 7$ appears stable and shows meaningful structure in the data, although the presence of multiple local optima shows that the result is still sensitive to initialization.

5.2.2 GMM

We used Gaussian Mixture Models (GMMs) as a probabilistic comparison to k-means. To select the number of components, we evaluated models using average BIC and the silhouette score. After fitting GMMs for $k \in [3, 100]$, the BIC curve showed a slight elbow in the range 40 to 60, with a subtle peak in the silhouette score in the same region. However, the silhouette values were low and noisy, staying mostly between 0.10 and 0.20. This behavior is consistent with our setting, since the inputs are high dimensional and mostly sparse due to many one-hot features, and GMMs allow overlapping clusters with soft boundaries while the silhouette is based on Euclidean distances. As a result, we treated the silhouette score as a secondary diagnostic and relied more on BIC, log-likelihood, and stability metrics. To choose a good k within the narrower range $k \in [40, 60]$, we ran each GMM configuration 20 times with different random initializations and plotted k versus variability of the final log-likelihoods and the mean pairwise Adjusted Rand Index (ARI) between runs. The ARI curve reached its maximum around $k = 51$, which also coincided with a favorable region of BIC and relatively low variability in log-likelihood, so we selected $k = 51$ as our final GMM setting.

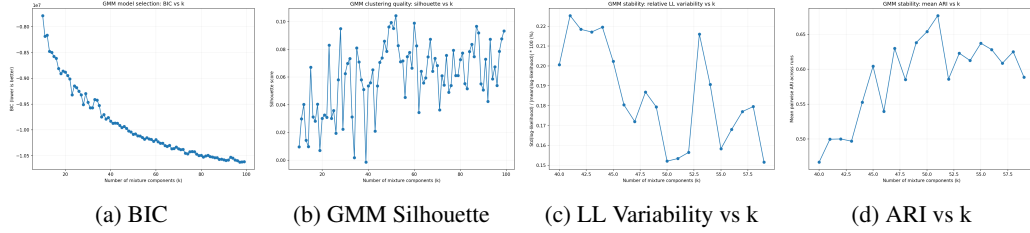


Figure 2: BIC, Silhouette, and Stability Plots

5.2.3 Results and Comparisons Between K-Means and GMM

When we inspected the cluster centroids, some features, such as release year and price, showed clear differences across clusters and were relatively easy to interpret. In contrast, features represented by sparse multihot encodings (for example, genres and categories) were harder to read directly from the centroid vectors. To better understand the structure, we manually inspected the clusters with two-dimensional visualizations. We applied PCA to project the feature vectors to two dimensions and colored each point by its cluster assignment. This allowed us to see how well-separated the clusters were in a lower-dimensional embedding and to inspect characteristic games within each group.

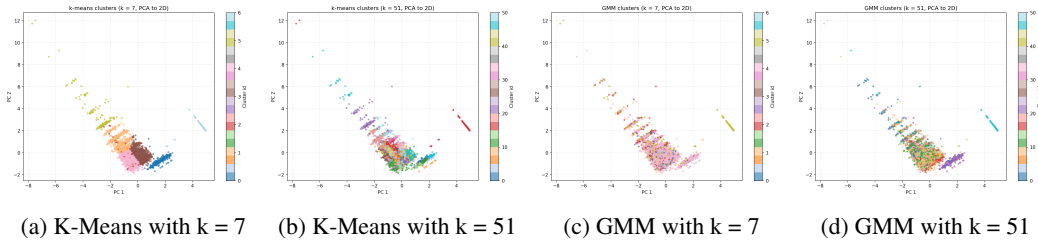


Figure 3: K-means vs GMM Results

The optimal values of k for k-means and GMM were quite different: k-means worked best with a small number of clusters ($k = 7$), while GMM preferred many more components ($k = 51$). This is reasonable given the modeling assumptions. K-means assigns each game to exactly one cluster based on distance, which tends to produce a few broad groups. GMM assumes that points are generated from a mixture of Gaussian distributions and gives each point soft membership in multiple components, so it can improve likelihood by using several components to cover complicated or skewed regions of the data. In our setting, the feature space is high dimensional and sparse due to many multi-hot features, and the true distribution of games is unlikely to match a small number of simple Gaussians, so it is not surprising that GMM uses more components than k-means.

GMM Cluster 1	GMM Cluster 2	K-Means Cluster 1	K-Means Cluster 2
Still Life	The Tower Of Elements	Team Fortress 2	Lootfest
Still Life 2	Star Chronicles: Delta Quadrant	Champions Online	3D Paraglider
Syberia	Project Druid - 2D Labyrinth	America's Army 3	Butsbal
Syberia II	Explorer-Stellar 2D	Pirates Vikings & Knights II	Radiator 2: Anniversary Edition
Nostradamus: The Last Prophecy	Eaten Alive	DC Universe Online	Sound Shift
Dracula: The Resurrection	Clergy Splode	Puzzle Pirates	Absent
Dracula 2: The Last Sanctuary	Ampersand	Fallen Earth Classic	Ultimate Rock Crawler
Dracula 3: The Path of the Dragon	Lup	America's Army: Proving Grounds	Tabletopia
Iron Storm	Turbo Pug	Crusader Kings II	Carpe Diem
The Cameron Files: The Secret at Loch Ness	Kivi, Toilet and Shotgun	AirMech	Rytmik Player

Table 3: Example games from four discovered clusters (10 per cluster).

The PCA visualizations reflect this difference. At $k = 7$, the k-means clusters are distinct in the two-dimensional PCA projection, while the GMM assignments look more mixed in the same space. At $k = 51$, both methods produce finer-grained partitions that look reasonable. Because the dataset is large, it is hard to inspect every cluster, but at $k = 51$ the clusters are small enough that we can examine the individual games in each cluster and find distinct patterns.

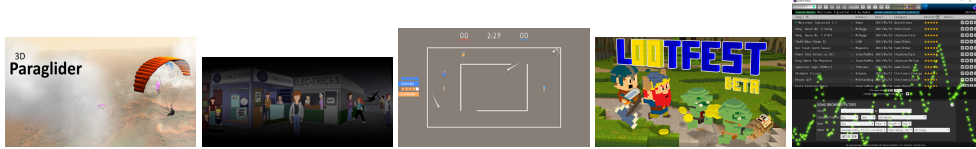


Figure 4: K-Means Cluster 2 (Free Low-Quality Indie Games)

GMM Cluster 1 consisted almost entirely of Microids games, which is consistent with the fact that we included the developer as a feature. GMM Cluster 2 grouped together small \$1.99 indie titles on Steam that were often tied to the publisher Back To Basics Gaming and appeared in large shovelware bundles from around 2015 to 2016. K-means Cluster 1 contained live-service, free-to-play free base games. K-means Cluster 2 contained free, short-form indie, or experimental titles that look more like prototypes than full free-to-play services.

We did not expect perfectly clean clusters, since the game space is large and closer to continuous than discrete, and we initially thought clusters would be driven mainly by a single feature such as release year or genre. Instead, the discovered clusters reflect combinations of features like price, genre, categories, and publisher, and sometimes capture higher-level properties such as business model or purpose. For example, being “shovelware” is not directly encoded as a feature, but emerges from the joint pattern of very low price, specific publishers, and particular genre and category combinations.

6 Conclusion / Future Work

Our report shows that static Steam metadata can be used to predict and analyze game popularity via a composite score combining ownership, playtime, recent activity, review sentiment, Metacritic ratings, and discounts. Among supervised models, the feedforward neural network was the highest-performing, achieving slightly lower validation MSE than linear regression (approximately 0.47 vs. 0.48); we believe the improvement is modest because many features already encode strong domain signals and the popularity target is noisy, so there is limited additional nonlinearity to exploit. For clustering, k-means with $k = 7$ produced the clearest, most stable, and most interpretable groups, while GMM with $k = 51$ fit the data more flexibly and uncovered finer-grained niches but at the cost of more overlapping and harder-to-interpret clusters. If we had more time and resources, we would (i) benchmark against stronger supervised baselines such as tree-based ensembles and gradient boosting, (ii) refine the popularity target to include temporal and social signals rather than a single static score, (iii) explore clustering methods and distance measures tailored to sparse multi-hot features, and (iv) perform deeper data cleaning and cross-platform enrichment to reduce noise and broaden coverage.

Note: All sections before this point must fit on five (5) pages. No exceptions. Supplemental material is not allowed. Anything else you want to add to your report (e.g. acknowledgements, author bios, funding sources) is included in the 5 page limit. The exception is the section describing the contributions of each team member. You will be penalized -10 points per page exceeding this limit. The max report score is 100.

7 Appendices

This section is optional for theory/algorithmic projects. Include additional derivations of proofs which weren't core to the understanding of your proposed algorithm in the methods section.

8 Contributions

- **Kyle Huang:** Focused more on Linear Regression and Neural Network implementation and analysis. Equal amounts of dataset gathering and processing and writing for proposal, milestone, and final project.
- **Alexander Zhong:** Focused on GMM and K-means implementation and analysis. Equal amounts of dataset gathering and processing and writing for proposal, milestone, and final project.

9 Note on citations

There are two citation commands:

Citation in parentheses `\citep{}` is for when you cite a paper at the end of a clause, and you could read the text out loud and not read the authors' names. For example "We also run our experiments on a multilingual language model (?)."

Citation in text `\citet{}` is for in-text citations, when someone reading the text out loud would have to read the names of the authors for the sentence to make sense. For example, "We also run our experiments on the multilingual model described in ?."

References

Oleg Chertov and Valerii Buslaiev. 2025. Video game sales prediction based on social media data using machine learning: A survey and future directions. *International Journal of Information Technology and Computer Science*, 17(4):49–57.

Valve Corporation. Steamworks web api documentation. <https://steamcommunity.com/dev>. Accessed: November 2025.

Nik Davis. 2021. Steam store games dataset. <https://www.kaggle.com/datasets/nikdavis/steam-store-games?resource=download>. Accessed: November 2025.

Matthew Horsfall and Andreas Oikonomou. 2011. A study of how different game play aspects can affect the popularity of role-playing video games. In *2011 16th International Conference on Computer Games (CGAMES)*, pages 63–69. IEEE.

Wenxiang Hu, Yiming Wang, and Ruoming Xia. 2024. Machine learning-based steam platform game's popularity analysis. In *Proceedings of the 1st International Conference on Data Science and Engineering (ICDSE 2024)*, pages 582–588.

Ziyang Jiang. 2021. Predicting the popularity of independent video games on the steam platform. Master's paper, University of North Carolina at Chapel Hill.

Jianbin Li, Yufan Zheng, Haoran Hu, Junhui Lu, and Choujun Zhan. 2021. Predicting video game sales based on machine learning and hybrid feature selection method. In *Proceedings of the 2021 16th International Conference on Intelligent Systems and Knowledge Engineering (ISKE)*, pages 497–502.

- Andraž De Luisa, Jan Hartman, David Nabergoj, Samo Pahor, Marko Rus, Bozhidar Stevanoski, Jure Demšar, and Erik Strumbelj. 2021. Predicting the popularity of games on steam. *Elektrotehniški Vestnik*, 88(4):151–162.
- Jan Zdražil Lukáš Križ and Jan Vašíček. 2024. Predicting the number of video game players: Application of machine learning methods. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 15(12):321–329. Accessed: November 2025.
- SteamSpy. Steamspy api documentation. <https://steamspy.com/api.php>. Accessed: November 2025.